

1. Arquitetura e Stack Tecnológica

A arquitetura do projeto adota uma abordagem moderna baseada em **Monorepo**, garantindo partilha eficiente de contratos e integração contínua unificada. O sistema foca em modularidade rígida no backend e alta performance de indexação (SEO) no frontend.

CAMADA / DOMÍNIO	TECNOLOGIA SELECIONADA	JUSTIFICATIVA / PAPEL NO ECOSISTEMA
Estratégia de Repositório	Monorepo	Código backend e frontend residem no mesmo repositório, facilitando a rastreabilidade de novas funcionalidades (commits unificados), gestão de dependências e sincronização de contratos de API.
Backend (API)	Kotlin + Spring Boot + Spring Modulith	O uso do Spring Modulith estabelece um <i>monólito modular</i> . Garante o isolamento estrito de domínios (Domain-Driven Design) e comunicação orientada a eventos internos, evitando a complexidade de microsserviços no MVP, mas permitindo extração fácil no futuro.
Frontend (Web)	Angular SSR + TailwindCSS + AngularMaterial	A adoção de <i>Server-Side Rendering (SSR)</i> no Angular é vital para indexação (SEO) e pré-visualização em redes sociais. TailwindCSS atua nos layouts rápidos, e o Angular Material fornece componentes de interface complexos.
Banco de Dados	PostgreSQL	Responsável pela persistência primária da aplicação. Selecionado por sua robustez em conformidade ACID e capacidade avançada de indexação.
Armazenamento	MinIO	Servidor de objetos compatível com a API S3. Atuará no armazenamento independente de todas as mídias e imagens processadas pela plataforma.

CAMADA / DOMÍNIO	TECNOLOGIA SELECIONADA	JUSTIFICATIVA / PAPEL NO ECOSISTEMA
Infraestrutura	Docker + Docker Compose	Empacotamento completo do sistema num ecossistema imutável, garantindo paridade entre desenvolvimento, testes e produção.

2. Convenções de Código (Coding Guidelines)

2.1. Versionamento (Git e Monorepo)

- **Estratégia:** *Trunk-based Development* adaptado para monorepo. Commits na branch principal (`main`) devem, idealmente, abranger alterações consistentes tanto no diretório do backend quanto no do frontend, se a funcionalidade assim o exigir.
- **Padrão de Commits:** Uso de **Conventional Commits**, recomendando a inclusão do escopo (ex: `feat(api): adiciona validacao no editor` ou `fix(web): corrige renderizacao SSR do post`).

2.2. Padrões de Backend (Spring Modulith)

- **Arquitetura de Módulos:** O código deve ser organizado por domínios de negócio (ex: `content` , `identity` , `ai-integration`) e não por camadas técnicas genéricas na raiz.
- **Isolamento (Encapsulamento):** Classes de lógica interna de um módulo não devem ser expostas (uso de `internal` no Kotlin ou visibilidade de pacote). O Spring Modulith testará e falhará o build caso ocorram dependências cíclicas ou quebra de encapsulamento entre módulos.
- **Comunicação Interna:** Interações transversais entre módulos devem, preferencialmente, utilizar a publicação de eventos (Spring Application Events) para reduzir o acoplamento direto.
- **Linting:** Utilização do `ktlint` como formatador oficial.

2.3. Padrões de Frontend (Angular SSR)

- **Renderização (SSR/Prerendering):** Rotas públicas (portefólio, artigos) devem priorizar o render no servidor ou pré-renderização no momento do build. Rotas administrativas podem utilizar renderização no lado do cliente (CSR).
- **Arquitetura Angular:** Privilegiar *Standalone Components* para reduzir o jargão de NgModules, mantendo a árvore de dependências limpa e o *bundle* final otimizado.
- **Estilização:** TailwindCSS para o layout geral; Angular Material reservado para interações complexas (Modais, Seletores).

2.4. Padrões de Banco de Dados

- **Nomenclatura:** Uso de `snake_case`, no plural (Ex: `users`, `blog_posts`).
- **Migrações:** Obrigatório o uso de *Flyway* ou *Liquibase* para versionamento do esquema de dados.